Top-Down Parsing
LL(1) Parser

Predictive Top-Down Parsers

- Like recursive-descent but parser can “predict” which production to use
  - By looking at the next few tokens
  - No backtracking

- Predictive parsers accept LL(k) grammars
  - L means “left-to-right” scan of input
  - L means “leftmost derivation”
  - k means “predict based on k tokens of lookahead”
  - In practice, LL(1) is used

Recursive Descent vs. LL(1)

- In recursive-descent,
  - At each step, many choices of production to use
  - Backtracking used to undo bad choices
- In LL(1),
  - At each step, only one choice of production
  - That is
    - When a non-terminal A is leftmost in a derivation
    - And the next input symbol is t
    - There is a unique production $A \rightarrow \alpha$ to use
      - Or no production to use (an error state)
- LL(1) is a recursive descent variant without backtracking

Predictive Parsing and Left Factoring

- Recall the grammar

  $E \rightarrow T + E \mid T$

  $T \rightarrow int \mid int * T \mid (E)$

- Hard to predict because
  - For T two productions start with int
  - For E it is not clear how to predict

- We need to <u>left-factor</u> the grammar

Left-Factoring Example

- Recall the grammar

  $E \rightarrow T + E \mid T$

  $T \rightarrow \text{int} * T \mid \text{int} \mid (E)$

- Factor out common prefixes of productions

  $E \rightarrow T X$ $\quad$ $X \rightarrow + E \mid \varepsilon$

  $T \rightarrow \text{int } Y \mid (E)$ $\quad$ $Y \rightarrow * T \mid \varepsilon$

LL(1) Parsing Table Example

- Left-factored grammar

  $E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

  $T \rightarrow \text{int}\ Y \mid (\ E\ )$ $\qquad$ $Y \rightarrow *\ T \mid \varepsilon$

- The LL(1) parsing table:

next input token

	int	*	+	(	)	$
E	T X			T X		
X			+ E		ε	ε
T	int Y			(E)		
Y		* T	ε		ε	ε

leftmost non-terminal

rhs of production to use

$E \rightarrow T\ X$ $\quad$ $X \rightarrow +\ E \mid \varepsilon$

$T \rightarrow int\ Y \mid (\ E\ )$ $\quad$ $Y \rightarrow *\ T \mid \varepsilon$

LL(1) Parsing Table Example

- Consider the [E, int] entry
  - "When current non-terminal is E and next input is int, use production $E \rightarrow T\ X$"
  - This can generate an int in the first position

	int	*	+	(	)	$
E	T X			T X		
X			+ E		ε	ε
T	int Y			(E)		
Y		* T	ε		ε	ε

$E \rightarrow T\ X$ $\quad$ $X \rightarrow +\ E \mid \varepsilon$

$T \rightarrow \text{int}\ Y \mid (\ E\ )$ $\quad$ $Y \rightarrow *\ T \mid \varepsilon$

LL(1) Parsing Tables. Errors

- Consider the [Y,+] entry
  - "When current non-terminal is Y and current token is +, get rid of Y"
  - Y can be followed by + only if $Y \rightarrow \varepsilon$

	int	*	+	(	)	$
E	T X			T X		
X			+ E		ε	ε
T	int Y			(E)		
Y		* T	ε		ε	ε

$E \rightarrow T\ X$ $\quad$ $X \rightarrow +\ E \mid \varepsilon$

$T \rightarrow \text{int}\ Y \mid (\ E\ )$ $\quad$ $Y \rightarrow *\ T \mid \varepsilon$

LL(1) Parsing Tables. Errors

- Consider the [Y,(] entry
  - "There is no way to derive a string starting with (from non-terminal Y"
  - Blank entries indicate error situations

	int	*	+	(	)	$
E	T X			T X		
X			+ E		ε	ε
T	int Y			(E)		
Y		* T	ε		ε	ε

Using Parsing Tables

- Method similar to recursive descent, except
  - For the leftmost non-terminal S
  - We look at the next input token a
  - And choose the production shown at [S,a]

- A stack records frontier of parse tree
  - Non-terminals that have yet to be expanded
  - Terminals that have yet to matched against the input
  - Top of stack = leftmost pending terminal or non-terminal

- Reject on reaching error state
- Accept on end of input & empty stack

LL(1) Parsing Algorithm (using the table)

```
initialize stack = <S $> and next
repeat
   case stack of
      <X, rest>   : if T[X,*next] = Y1…Yn
                        then stack ← <Y1…Yn, rest>;
                        else  error ();

      <t, rest>   : if t == *next ++
                        then  stack ← <rest>;
                        else error ();

until stack == < >
```

LL(1) Parsing Algorithm

$ marks bottom of stack

```
initialize stack = <S $> and next
repeat
   case stack of
      <X, rest>   : if T[X,*next] = Y1...Yn
                       then stack ← <Y1...Yn, rest>;
                       else  error ();

      <t, rest>   : if t == *next ++
                       then  stack ← <rest>;
                       else error ();
   until stack == < >
```

For non-terminal X on top of stack, lookup production

Pop X, push production rhs on stack. Note leftmost symbol of rhs is on top of the stack.

For terminal t on top of stack, check t matches next input token.

LL(1) Parsing Example

$E \rightarrow T\ X$ $\quad$ $X \rightarrow +\ E \mid \varepsilon$

$T \rightarrow \text{int}\ Y \mid (\ E\ )$ $\quad$ $Y \rightarrow *\ T \mid \varepsilon$

Stack	Input	Action
E $	int * int $	T X
T X $	int * int $	int Y
int Y X $	int * int $	terminal
Y X $	* int $	* T
* T X $	* int $	terminal
T X $	int $	int Y
int Y X $	int $	terminal
Y X $	$	ε
X $	$	ε
$	$	ACCEPT

Constructing Parsing Tables: The Intuition

- Consider non-terminal A, production $A \rightarrow \alpha$, and token t

Greek letters denote strings of non-terminals and terminals

1. Add $T[A,t] = \alpha$
   if $A \rightarrow \alpha \rightarrow^* t\ \beta$
   - α can derive a t in the first position
   - We say that $t \in First(\alpha)$

2. Add $T[A,t] = \varepsilon$
   if $A \rightarrow \alpha \rightarrow^* \varepsilon$ and $S \rightarrow^* \gamma\ A\ t\ \delta$
   - Useful if stack has A, input is t, and A cannot derive t
   - In this case only option is to get rid of A (by deriving ε)
     - Can work only if t can follow A in at least one derivation
   - We say $t \in Follow(A)$

Computing First Sets

Definition

$$First(X) = \{ t \mid X \rightarrow^* t\alpha \} \cup \{ \varepsilon \mid X \rightarrow^* \varepsilon \}$$

Algorithm sketch:

1. $First(t) = \{ t \}$
2. $\varepsilon \in First(X)$
   - if $X \rightarrow \varepsilon$ or
   - if $X \rightarrow A_1 \ldots A_n$ and $\varepsilon \in First(A_i)$ for all $1 \leq i \leq n$
3. $First(\alpha) \subseteq First(X)$
   - if $X \rightarrow \alpha$ or
   - if $X \rightarrow A_1 \ldots A_n \alpha$ and $\varepsilon \in First(A_i)$ for all $1 \leq i \leq n$

First Sets: Example

1. $First(t) = \{ t \}$
2. $\varepsilon \in First(X)$
   - if $X \rightarrow \varepsilon$ or
   - if $X \rightarrow A_1 \ldots A_n$ and $\varepsilon \in First(A_i)$ for all $1 \leq i \leq n$
3. $First(\alpha) \subseteq First(X)$
   - if $X \rightarrow \alpha$ or
   - if $X \rightarrow A_1 \ldots A_n\ \alpha$ and $\varepsilon \in First(A_i)$ for all $1 \leq i \leq n$

$E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

$T \rightarrow int\ Y \mid (\ E\ )$ $\qquad$ $Y \rightarrow\ *\ T \mid \varepsilon$

First(E) = $\qquad$ First(X) =

First(T) = $\qquad$ First(Y) =

First Sets: Example

- Recall the grammar

  $E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

  $T \rightarrow \text{int}\ Y \mid (\ E\ )$ $\qquad$ $Y \rightarrow *\ T \mid \varepsilon$

- First sets

  First(( ) = { (}

  First( )) = {) }

  First(int) = { int }

  First(+) = { + }

  First(*) = { * }

  First(T) = {int, (}

  First(E) = {int, (}

  First(X) = {+, ε }

  First(Y) = {*, ε }

Computing Follow Sets

- Definition:

  $$\text{Follow}(X) = \{ t \mid S \rightarrow^* \beta\, X\, t\, \delta \}$$

- Intuition
  - If $X \rightarrow A\ B$ then $\text{First}(B) \subseteq \text{Follow}(A)$ and
    $\text{Follow}(X) \subseteq \text{Follow}(B)$
    - if $B \rightarrow^* \varepsilon$ then $\text{Follow}(X) \subseteq \text{Follow}(A)$
  - If S is the start symbol then $\$ \in \text{Follow}(S)$

Computing Follow Sets (Cont.)

Algorithm sketch:

1. $\$ \in \text{Follow}(S)$
2. For each production $A \rightarrow \alpha X \beta$
   - $\text{First}(\beta) - \{\varepsilon\} \subseteq \text{Follow}(X)$
3. For each production $A \rightarrow \alpha X \beta$ where $\varepsilon \in \text{First}(\beta)$
   - $\text{Follow}(A) \subseteq \text{Follow}(X)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

  $E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

  $T \rightarrow (\ E\ ) \mid \text{int}\ Y$ $\qquad$ $Y \rightarrow *\ T \mid \varepsilon$

- $\$ \in \text{Follow}(E)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

  $E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

  $T \rightarrow (\ E\ ) \mid \text{int}\ Y$ $\qquad$ $Y \rightarrow *\ T \mid \varepsilon$

- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$ because $\varepsilon \in \text{First}(X)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

  $E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

  $T \rightarrow (\ E\ ) \mid \text{int}\ Y$ $\qquad$ $Y \rightarrow *\ T \mid \varepsilon$

- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$ because $\varepsilon \in \text{First}(X)$
- $) \in \text{Follow}(E)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

  $E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

  $T \rightarrow (\ E\ ) \mid \text{int}\ Y$ $\qquad$ $Y \rightarrow *\ T \mid \varepsilon$

- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$ because $\varepsilon \in \text{First}(X)$
- $) \in \text{Follow}(E)$
- $\text{Follow}(T) \subseteq \text{Follow}(Y)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

  $E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

  $T \rightarrow (\ E\ ) \mid \text{int}\ Y$ $\qquad$ $Y \rightarrow *\ T \mid \varepsilon$

- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$ because $\varepsilon \in \text{First}(X)$
- $) \in \text{Follow}(E)$
- $\text{Follow}(T) \subseteq \text{Follow}(Y)$
- $\text{Follow}(X) \subseteq \text{Follow}(E)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

  $E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

  $T \rightarrow (\ E\ ) \mid \text{int}\ Y$ $\qquad$ $Y \rightarrow *\ T \mid \varepsilon$

- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$ because $\varepsilon \in \text{First}(X)$
- $) \in \text{Follow}(E)$
- $\text{Follow}(T) \subseteq \text{Follow}(Y)$
- $\text{Follow}(X) \subseteq \text{Follow}(E)$
- $\text{Follow}(Y) \subseteq \text{Follow}(T)$

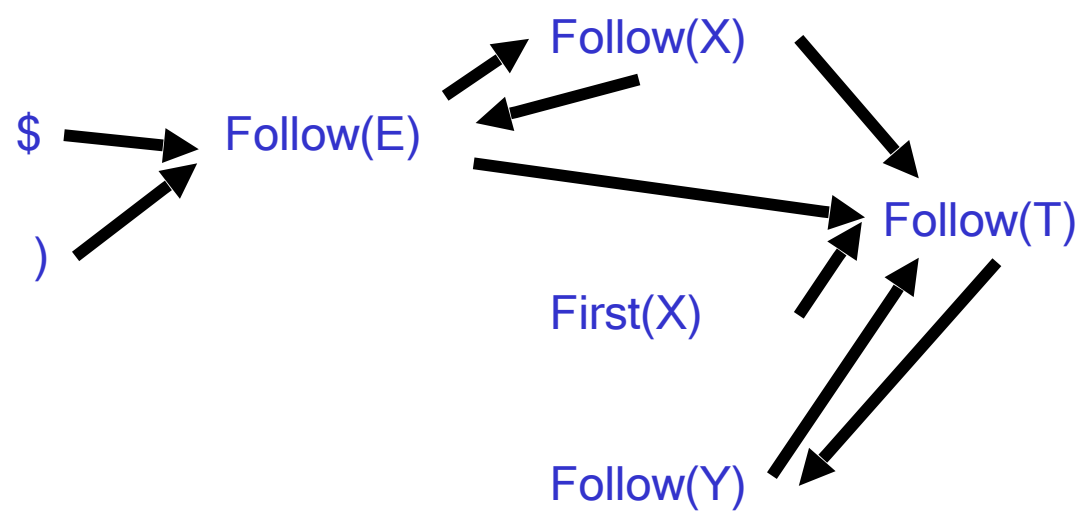
Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

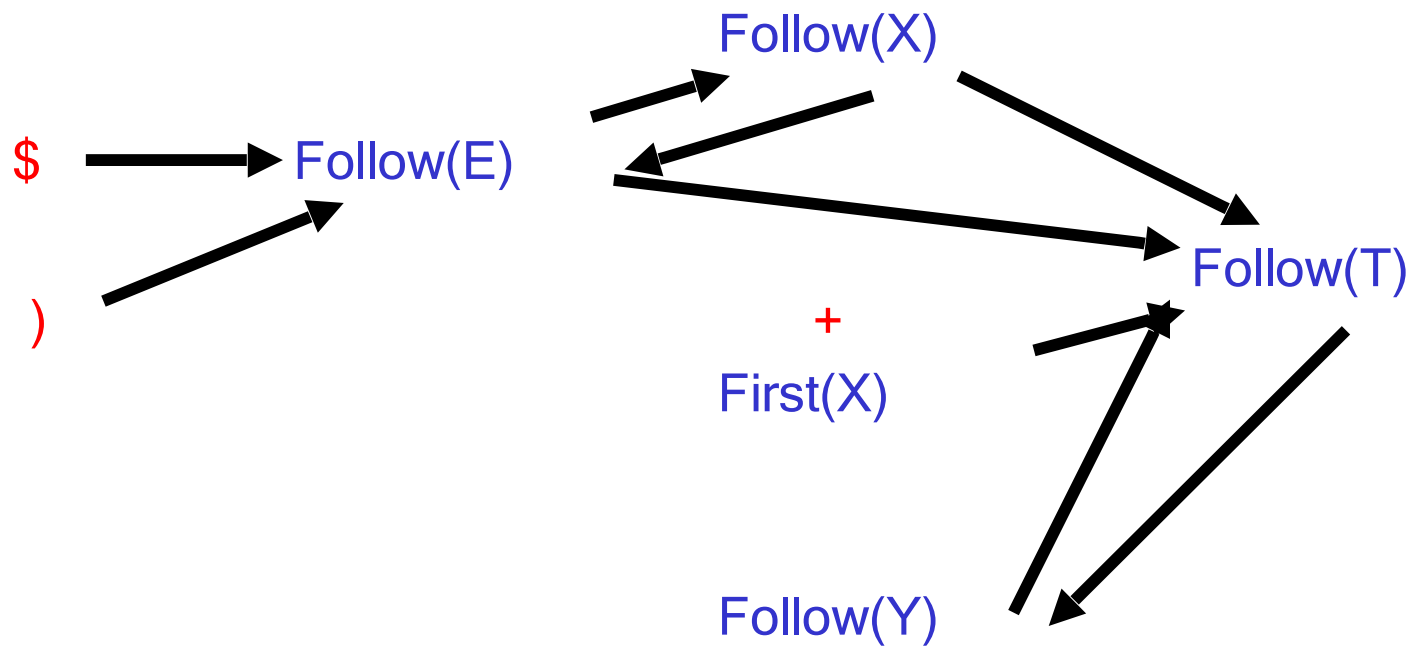
  $E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

  $T \rightarrow (\ E\ ) \mid \text{int}\ Y$ $\qquad$ $Y \rightarrow *\ T \mid \varepsilon$

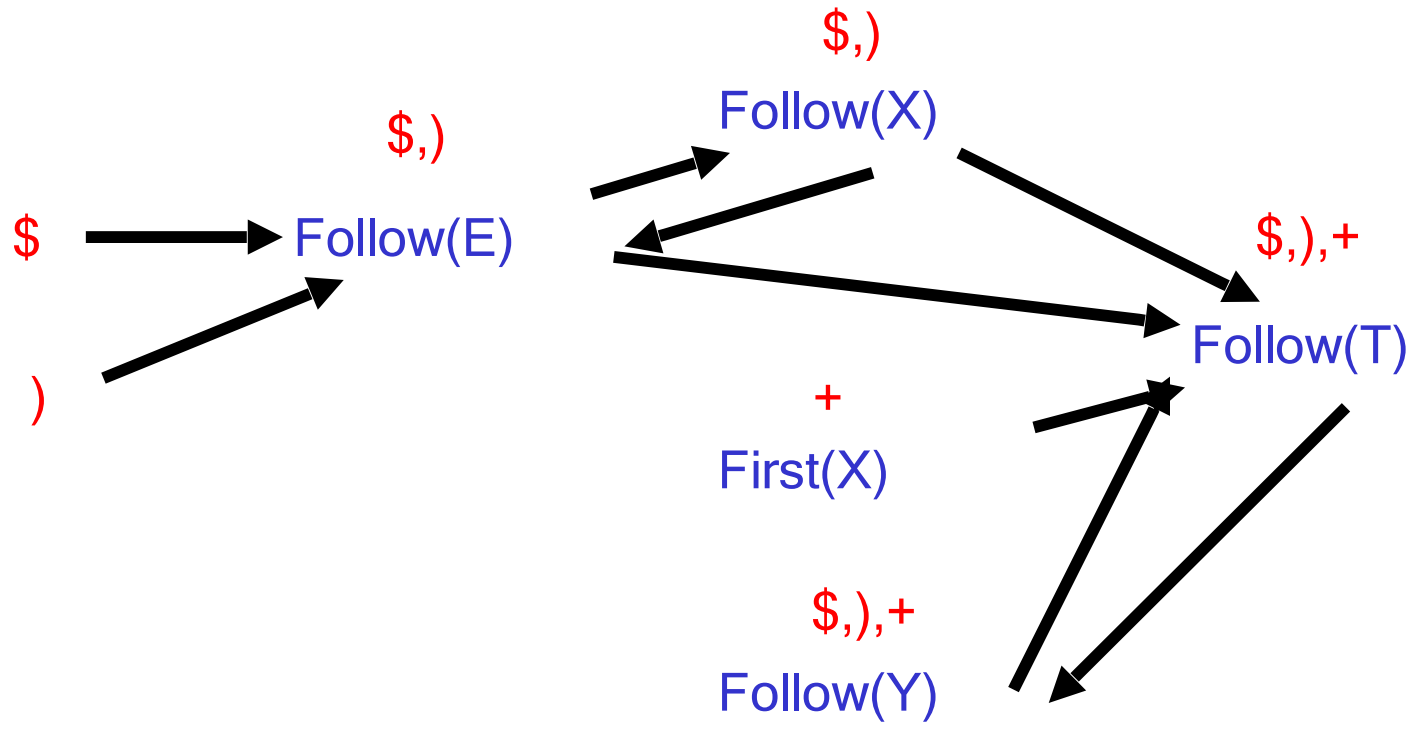
- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$
- $) \in \text{Follow}(E)$
- $\text{Follow}(T) \subseteq \text{Follow}(Y)$
- $\text{Follow}(X) \subseteq \text{Follow}(E)$
- $\text{Follow}(Y) \subseteq \text{Follow}(T)$



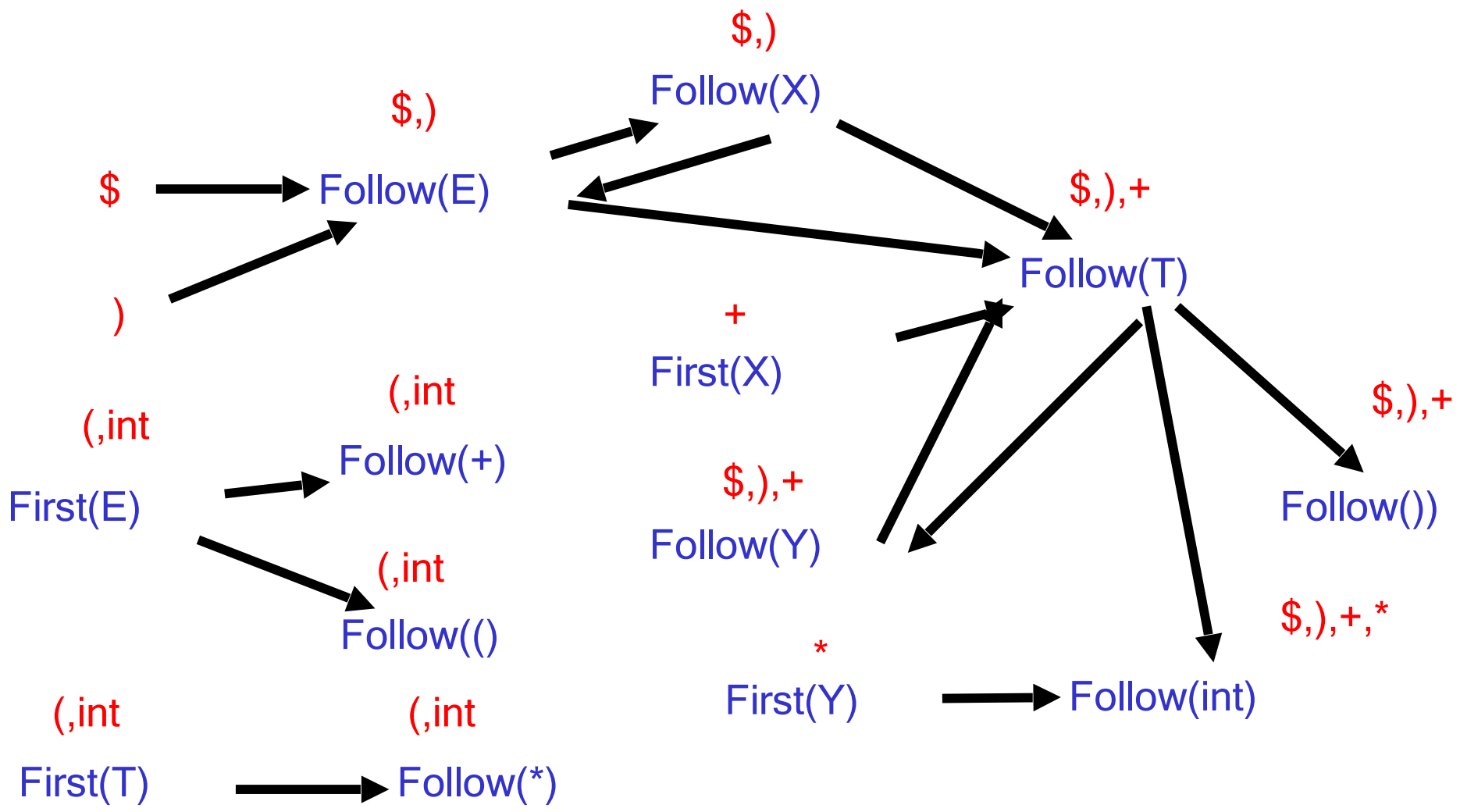
Computing the Follow Sets (for the Non-Terminals)



Computing the Follow Sets (for the Non-Terminals)



Computing the Follow Sets (for all symbols)



Follow Sets: Example

- Recall the grammar

  $E \rightarrow T\ X$ $\qquad$ $X \rightarrow +\ E \mid \varepsilon$

  $T \rightarrow (\ E\ ) \mid \text{int}\ Y$ $\qquad$ $Y \rightarrow\ *\ T \mid \varepsilon$

- Follow sets

  Follow(+) = { int, (} $\qquad$ Follow(*) = {int, (}

  Follow(( ) = { int, (} $\qquad$ Follow(E) = {$,)}

  Follow(X) = {$,) } $\qquad$ Follow(T) = {$, +,)}

  Follow( )) = {+,) , $} $\qquad$ Follow(Y) = {$, +,)}

  Follow(int) = {*, +,) , $}

Constructing LL(1) Parsing Tables

- Construct a parsing table T for CFG G

- For each production $A \rightarrow \alpha$ in G do:
  - For each terminal $t \in First(\alpha)$ do
    - $T[A, t] = \alpha$
  - If $\varepsilon \in First(\alpha)$, then for each $t \in Follow(A)$ do
    - $T[A, t] = \varepsilon$
  - If $\varepsilon \in First(\alpha)$ and $\$ \in Follow(A)$ do
    - $T[A, \$] = \varepsilon$

Notes on LL(1) Parsing Tables

- If any entry is multiply defined then G is not LL(1)
  - If G is ambiguous
  - If G is left recursive
  - If G is not left-factored
  - <u>And in other cases as well</u>